

HOIL

Sistema de Inventario

Documentación técnica completa

Cristian Alejandro Hoil Reyes

CECYTE — Taller de Refrigeración

2026

Índice general

1	Resumen	5
1.1	Propósito	5
1.2	Objetivo funcional	5
1.3	Alcance	5
1.4	Lecturas complementarias	6
2	Stack, arquitectura y estructura	7
2.1	Stack tecnológico	7
2.2	Justificación de la arquitectura	8
2.3	Decisión arquitectónica principal	8
2.4	Diagrama de arquitectura	9
2.5	Flujo de datos	9
2.6	Rutas del App Router	10
2.7	Organización de directorios	10
2.8	Mapa de archivos principales	12
2.9	Convenciones técnicas	15
3	Autenticación y roles	16
3.1	Métodos de autenticación	16
3.2	Modelo de sesión	16
3.3	Flujo de autenticación	17
3.4	Roles del sistema	17
3.5	Protección de rutas por rol	17
3.6	Creación de usuarios	18
3.7	Restablecimiento de contraseña	18
3.8	Consideraciones de operación para Google OAuth	18
4	Flujos funcionales y reglas de negocio	20
4.1	Flujo de préstamo de material	20
4.2	Flujo de devolución de material	21

4.3	Flujo de consumo de material	21
4.4	Flujo de entrada de stock (Admin)	22
4.5	Creación y edición de materiales (Admin)	22
4.5.1	Creación	22
4.5.2	Edición	23
4.5.3	Imagen guía	23
4.5.4	Eliminación lógica	23
4.6	Gestión de equipos (Admin)	23
4.7	Gestión de usuarios (Admin)	24
4.8	Reglas de negocio	24
5	Modelo de datos	25
5.1	Diagrama Entidad-Relación	25
5.2	Tabla <code>perfiles</code>	25
5.3	Tabla <code>equipos</code>	26
5.4	Tabla <code>materiales</code>	26
5.5	Tabla <code>movimientos</code>	27
5.6	Tabla <code>prestamos</code>	28
5.7	Relaciones entre tablas	30
5.8	Categorías de materiales	30
5.9	Estados de material	30
5.10	Tipos de movimiento	31
5.11	Stock semilla	31
6	Supabase, migraciones, operación y pruebas	32
6.1	Supabase local	32
6.2	Migraciones	32
6.3	Supabase Storage	32
6.4	Variables de entorno	33
6.5	Scripts operativos	33
6.6	Seed y carga inicial	33
6.7	Pruebas	33
6.8	Reportes (Admin)	34
7	UI/UX y diseño	35
7.1	Identidad visual institucional	35
7.2	Stack de diseño	36
7.3	Directrices de interfaz	36
7.3.1	Botones y acciones	36
7.3.2	Formularios	36

7.3.3	Regla de 3 clics	37
7.3.4	Diseño responsive	37
7.4	Distribución del Dashboard (Admin)	37
7.5	Panel del Representante	38
7.6	Actualización en tiempo real	38
7.7	Formulario de login	39
7.8	Decisiones UX relevantes	39
7.9	Componentes reutilizables	40
8	Componentes	41
8.1	Componente <code>RealtimeRefresh</code>	41
8.2	Componente <code>SearchBox</code>	42
8.3	Componentes de formularios	42
8.4	Componentes del dashboard	43
8.5	Componentes UI (shadcn/ui)	43
8.6	Clientes Supabase	44
8.7	Consideraciones de diseño	44
9	Anexos	46
9.1	Observaciones importantes	46
9.2	Decisiones de diseño del documento	46

Resumen

1.1 Propósito

Inventario HOIL es un sistema web para administrar materiales, equipos, usuarios y movimientos del taller de refrigeración del CECYTE. La aplicación está pensada para dos perfiles operativos: **administrador** y **representante de equipo**.

1.2 Objetivo funcional

El sistema centraliza el control de inventario, préstamos, devoluciones, consumos y reportes con trazabilidad completa. Además, ofrece actualizaciones en vivo mediante Supabase Realtime para que los cambios de stock se reflejen sin recargar manualmente la interfaz.

1.3 Alcance

El alcance actual cubre:

- autenticación por correo/contraseña y Google OAuth;
- dashboard administrativo con indicadores y gráficos;
- CRUD de materiales, equipos y usuarios;
- préstamos, devoluciones y consumo de materiales;

- reportes filtrados por fecha, tipo y búsqueda textual;
- pruebas unitarias y pruebas end-to-end.

1.4 Lecturas complementarias

La carpeta `docs/` contiene documentación previa en Markdown. Esta versión L^AT_EX la sintetiza y la organiza para revisión técnica:

- `00_stack_y_arquitectura.md`
- `01_contexto_y_objetivos.md`
- `02_requerimientos_y_rolles.md`
- `03_catalogo_materiales.md`
- `04_ui_ux_y_diseno.md`
- `05_plantilla_generador_manual.md`
- `06_modelo_base_datos.md`
- `07_kpis_y_dashboard.md`
- `08_estado_funcional_actual.md`
- `09_google_oauth.md`

Stack, arquitectura y estructura

2.1 Stack tecnológico

Cuadro 2.1: Stack tecnológico completo

Capa	Tecnología	Versión / Nota
Frontend	Next.js (App Router)	14.2.35, React 18
Hosting	Vercel	Free tier (100 GB/mes)
Backend / BD	Supabase	PostgreSQL gestionado + Auth + Realtime
ORM	Drizzle ORM	0.45.2, tipado, migraciones
Estilos	Tailwind CSS	TypeScript 3.4.1, utility-first
Componentes UI	shadcn/ui (Radix)	<code>components.json</code> , base-nova style
Iconos	Lucide React	1.14.0
Gráficos	Recharts	3.8.1, declarativo, nativo React
Lenguaje	TypeScript	5.x, <code>strict</code> mode
Gestor paquetes	pnpm (Corepack)	9.15.9
Pruebas unit.	Vitest	4.1.6, jsdom
Pruebas E2E	Playwright	1.60.0, Chromium + Mobile
Repositorio	GitHub	CI/CD automático a Vercel

2.2 Justificación de la arquitectura

La elección de Next.js + Supabase sobre alternativas como Laravel se fundamenta en tres criterios principales:

1. **Costo \$0 total:** Vercel y Supabase en sus tiers gratuitos cubren holgadamente el uso esperado del taller (aproximadamente 50 alumnos). No hay servidor que administrar, ni Apache, PHP o SSL que configurar.
2. **Tiempo real nativo:** Supabase Realtime permite recibir actualizaciones de inventario sin refrescar la página. En Laravel esto requeriría WebSockets, Echo y Redis adicionales.
3. **Google OAuth integrado:** Supabase Auth incluye el proveedor Google sin paquetes externos adicionales.

2.3 Decisión arquitectónica principal

El proyecto evita una API REST intermedia. Las páginas del App Router leen datos directamente desde Supabase en componentes servidor, mientras que las mutaciones se concentran en Server Actions. Esto reduce capas innecesarias y deja la lógica de permisos en un punto muy claro.

2.4 Diagrama de arquitectura

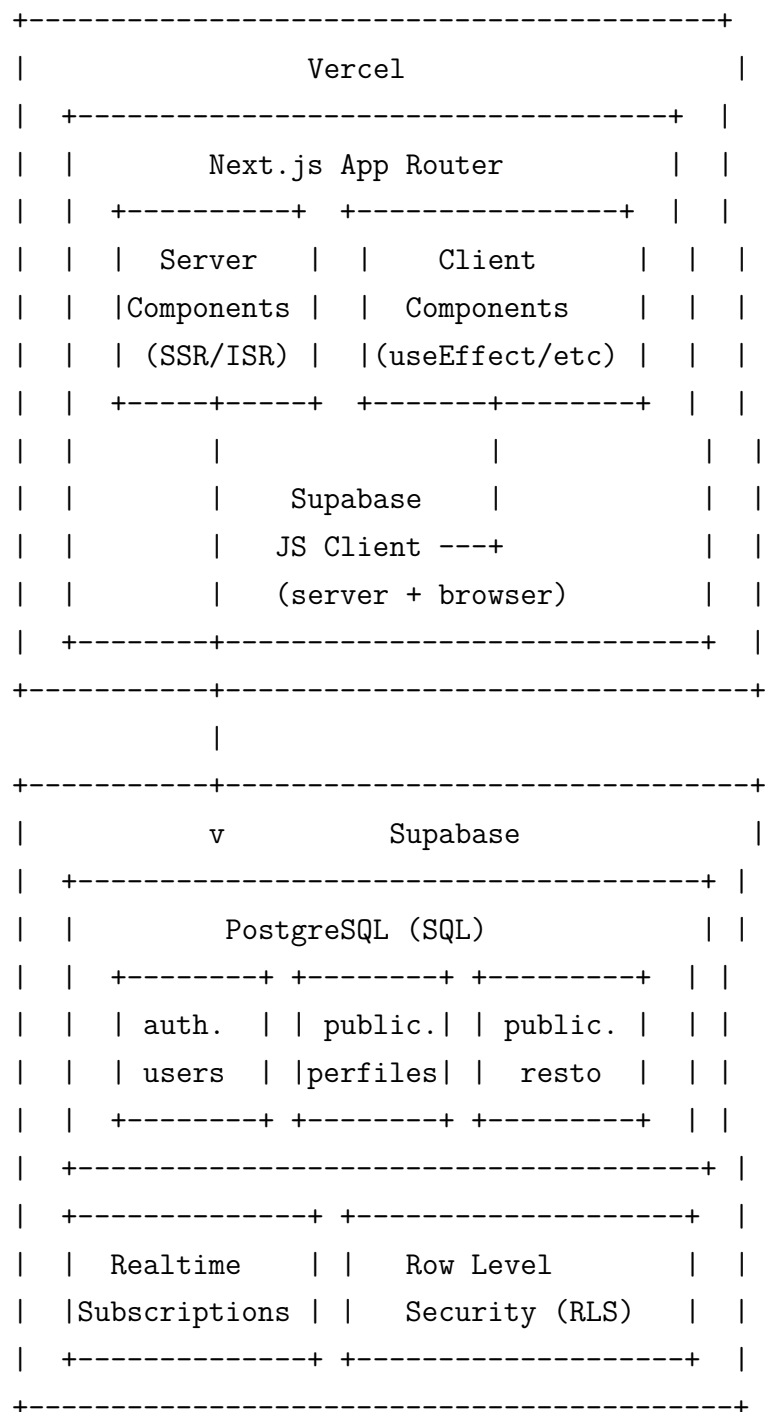


Figura 2.1: Arquitectura del sistema: Next.js en Vercel + Supabase

2.5 Flujo de datos

1. El usuario hace login a través del formulario de credenciales o Google OAuth. Supabase Auth devuelve un JWT y el `auth.users.id`.

2. El middleware de Next.js (`src/middleware.ts`) verifica la sesión y el rol en cada petición, redirigiendo según corresponda.
3. Los Server Components leen datos directamente de Supabase sin exponer la base de datos al cliente.
4. Los Client Components usan Supabase Realtime para recibir actualizaciones en vivo (por ejemplo, cambios de stock mientras otro representante solicita material).
5. Las mutaciones (crear préstamo, devolver, consumir) son Server Actions de Next.js que escriben a Supabase usando el cliente `service_role` para evitar bloqueos de RLS. Realtime notifica a los demás clientes tras cada cambio.

2.6 Rutas del App Router

Cuadro 2.2: Endpoints principales

Ruta	Propósito
/	Redirección al dashboard según rol
/login	Login con correo/contraseña y Google OAuth
/auth/callback	Callback OAuth, intercambio de código
/auth/signout	Cierre de sesión
/dashboard	Admin: KPIs y resumen
/dashboard/materiales	Admin: catálogo, stock, imágenes, eliminación
/dashboard/equipos	Admin: gestión de equipos
/dashboard/usuarios	Admin: perfiles, restablecimiento de contraseña
/dashboard/reportes	Admin: reportes filtrados, exportación CSV
/equipo	Representante: panel principal
/equipo/pedir	Representante: solicitar préstamo
/equipo/devolver	Representante: devolver material
/equipo/consumo	Representante: reportar consumo
/equipo/historial	Representante: historial del equipo

2.7 Organización de directorios

```
Proyecto_Hoil/
+-- src/
|   +-- app/                # Next.js App Router
```

		+++ (auth)/login/	# Página de login
		+++ (dashboard)/	# Shell y páginas del administrador
		+++ layout.tsx	# Sidebar + bottom tabs + signout
		+++ dashboard/	# KPIs y panel principal
		+++ materiales/	# Catálogo CRUD + imágenes
		+++ equipos/	# Gestión de equipos
		+++ usuarios/	# Perfiles + reset password
		+++ reportes/	# Reportes filtrados
		+++ (equipo)/	# Shell y páginas del representante
		+++ layout.tsx	# Header + info equipo + signout
		+++ equipo/	# Pedir, devolver, consumir
		+++ auth/callback/	# Callback OAuth
		+++ auth/signout/	# Ruta de cierre de sesión
		+++ page.tsx	# Landing: redirección por rol
		+++ layout.tsx	# Layout raíz
	+++ components/		
		+++ ui/	# Componentes shadcn/ui (button, card, ...)
		+++ dashboard/	# Componentes específicos del dashboard
		+++ forms/	# Formularios reutilizables
		+++ realtime-refresh.tsx	# Indicador de refresco en tiempo real
		+++ search-box.tsx	# Búsqueda con debounce
	+++ lib/		
	+++ actions/		
		+++ admin.ts	# Server Actions del administrador
		+++ prestamos.ts	# Server Actions de préstamos
	+++ db/		
		+++ schema.ts	# Esquema Drizzle ORM + relaciones
		+++ queries.ts	# Funciones KPI y consultas
		+++ seed.ts	# Datos semilla
	+++ domain/		
		+++ inventory.ts	# Reglas de negocio puras
	+++ supabase/		
		+++ server.ts	# Clientes Supabase (server + admin)
		+++ client.ts	# Cliente Supabase para navegador
	+++ utils.ts		# Utilidades generales
+++ supabase/			
	+++ migrations/		# Migraciones SQL (0001-0004)
+++ tests/			
	+++ unit/		

```

|   |   +-- inventory.test.ts      # Unit tests de reglas de dominio
|   +-- e2e/                      # Tests E2E con Playwright
|   +-- setup.ts                  # Setup global de Vitest
+-- docs/                        # Documentación complementaria (00-09)
+-- package.json
+-- tsconfig.json
+-- next.config.mjs
+-- tailwind.config.ts
+-- vitest.config.ts
+-- playwright.config.ts
+-- drizzle.config.ts
+-- components.json              # Configuración de shadcn/ui
+-- README.md

```

2.8 Mapa de archivos principales

Archivo	Responsabilidad
<code>src/app/page.tsx</code>	Punto de entrada. Redirige usuarios autenticados a <code>/dashboard</code> o <code>/equipo</code> según el rol.
<code>src/app/(auth)/login/page.tsx</code>	Formulario de login con correo/contraseña y botón de Google OAuth. Validación client-side y server-side.
<code>src/app/auth/callback/route.ts</code>	Intercambia el código OAuth de Google por una sesión Supabase y redirige al usuario según su rol.
<code>src/app/(dashboard)/layout.tsx</code>	Shell del administrador: sidebar en desktop, bottom tab bar en móvil, navegación, nombre de usuario y botón de cierre de sesión. Incluye <code>RealtimeRefresh</code> .
<code>src/app/(equipo)/layout.tsx</code>	Shell del representante: header con nombre del equipo, cierre de sesión, indicador de tiempo real.
<code>src/app/(dashboard)/dashboard/page.tsx</code>	Dashboard del admin con 9 KPIs: total materiales, prestado vs disponible, stock bajo, top 5, equipos activos, movimientos, tasa devolución, estado materiales, consumibles agotados.
<code>src/app/(dashboard)/dashboard/materiales/page.tsx</code>	CRUD de materiales: búsqueda, filtro por categoría, carga de imágenes a Supabase Storage, entrada de stock, eliminación lógica.
<code>src/app/(dashboard)/dashboard/equipos/page.tsx</code>	CRUD de equipos: validación de dependencias antes de eliminar.

Archivo	Responsabilidad
<code>src/app/(dashboard)/dashboard/perfil/page.tsx</code>	CRUD de perfil: creación de cuentas con contraseña, restablecimiento de contraseña, eliminación con verificación de historial.
<code>src/app/(dashboard)/dashboard/movimientos/page.tsx</code>	Vista de reportes y filtros por período (hora, día, semana, mes), tipo de movimiento, equipo y material. Exportación a CSV.
<code>src/app/(equipo)/equipo/page.tsx</code>	Panel del representante: catálogo de materiales disponibles, préstamos activos, acciones de pedir, devolver y consumir.
<code>src/lib/actions/admin.ts</code>	Server Actions del admin: <code>createMaterial</code> , <code>updateMaterial</code> , <code>deleteMaterial</code> , <code>addMaterialStock</code> , <code>createEquipo</code> , <code>updateEquipo</code> , <code>deleteEquipo</code> , <code>createUsuario</code> , <code>updatePerfil</code> , <code>resetUsuarioPassword</code> , <code>deletePerfil</code> , <code>updateMaterialImage</code> , <code>deleteMaterialImage</code> . Todas verifican <code>requireAdmin()</code> .
<code>src/lib/actions/prestamos.ts</code>	Server Actions del representante: <code>pedirMaterial</code> , <code>devolverMaterial</code> , <code>consumirMaterial</code> . Verifican <code>requireRepresentante()</code> y validan disponibilidad con reglas de dominio.
<code>src/lib/db/schema.ts</code>	Esquema Drizzle ORM: tablas <code>equipos</code> , <code>perfiles</code> , <code>materiales</code> , <code>movimientos</code> , <code>prestamos</code> con checks, defaults y relaciones. Tipos inferidos exportados.
<code>src/lib/db/queries.ts</code>	Funciones KPI: <code>getKpiMaterialesTotal</code> , <code>getPrestadoVsDisponible</code> , <code>getKpiStockBajo</code> , <code>getTopMateriales</code> , <code>getEquiposActividad</code> , <code>getMovimientosPorPeriodo</code> , <code>getKpiTasaDevolucion</code> , <code>getEstadoMateriales</code> , <code>getConsumiblesAgotados</code> .
<code>src/lib/domain/inventory.ts</code>	Reglas de negocio puras: <code>calcularPrestadoPendiente</code> , <code>calcularDisponible</code> , <code>validarCantidadDisponible</code> , <code>validarDevolucionPendiente</code> , <code>esConsumible</code> .

Archivo	Responsabilidad
<code>src/lib/supabase/server.ts</code>	Dos clientes Supabase: <code>createClient()</code> con cookies de sesión para Server Components; <code>createAdminClient()</code> con <code>service_role</code> para mutaciones privilegiadas.
<code>src/lib/supabase/client.ts</code>	Cliente Supabase para el navegador: usa <code>createBrowserClient</code> de <code>@supabase/ssr</code> .
<code>src/middleware.ts</code>	Middleware de Next.js: refresca sesión Supabase, protege rutas por rol (admin no accede a <code>/equipo</code> , representante no accede a <code>/dashboard</code>), redirige usuarios no autenticados a <code>/login</code> .
<code>src/components/realtime-refresh.tsx</code>	Componente cliente que se suscribe a cambios en <code>materiales</code> y <code>prestamos</code> vía Supabase Realtime y llama a <code>router.refresh()</code> para actualizar Server Components.
<code>src/components/search-box.tsx</code>	Campo de búsqueda con debounce de 300ms, controlado mediante URL search params.
<code>supabase/migrations/0001_schema.ts</code>	Migración inicial: creación de tablas, índices, triggers <code>updated_at</code> .
<code>supabase/migrations/0002_rls.ts</code>	Políticas RLS por tabla y rol.
<code>supabase/migrations/0003_materials_active.ts</code>	Columnas <code>active</code> y <code>eliminado_at</code> para eliminación lógica de materiales.
<code>supabase/migrations/0004_materials_images.ts</code>	Columnas <code>image_url</code> e <code>imagen_path</code> y bucket <code>materiales</code> en Storage.
<code>tailwind.config.ts</code>	Configuración de Tailwind con colores institucionales CECYTE, variables CSS de shadcn/ui, tipografía Inter.
<code>tsconfig.json</code>	Configuración TypeScript con <code>strict: true</code> y alias <code>@/ → src/</code> .
<code>playwright.config.ts</code>	Playwright con proyectos Chromium desktop y Pixel 5 mobile, <code>baseURL</code> configurable y <code>webServer</code> automático.
<code>vitest.config.ts</code>	Vitest con entorno jsdom, plugin React, setup global, alias <code>@/</code> , cobertura v8 sobre <code>src/lib/domain/</code> .
<code>drizzle.config.ts</code>	Configuración de Drizzle Kit para migraciones: schema en <code>src/lib/db/schema.ts</code> , salida en <code>supabase/migrations/</code> .

Archivo	Responsabilidad
<code>components.json</code>	Configuración de shadcn/ui: estilo base-nova, server components, alias, colores neutral con CSS variables.

2.9 Convenciones técnicas

- pnpm se usa vía Corepack.
- El código acepta variables de Supabase nuevas y legacy por compatibilidad local.
- La disponibilidad de material no se almacena: se calcula.
- La bitácora de movimientos es inmutable y sirve como trazabilidad.

Autenticación y roles

3.1 Métodos de autenticación

El sistema soporta dos métodos de autenticación mediante Supabase Auth:

1. **Correo electrónico y contraseña:** método primario. Las cuentas son creadas por el administrador desde `Dashboard > Usuarios`, quien define una contraseña inicial. El representante puede cambiar su contraseña posteriormente.
2. **Google OAuth:** método secundario con cuenta institucional de CECYTE. Requiere que el usuario ya tenga un perfil en `public.perfiles` con el mismo correo. Si un usuario se autentica con Google pero no tiene perfil, la aplicación cierra la sesión y muestra un mensaje de acceso no habilitado.

3.2 Modelo de sesión

Supabase Auth gestiona la tabla `auth.users` automáticamente. La aplicación mantiene una tabla de dominio `public.perfiles` vinculada 1:1 por `id` a `auth.users.id`. Esta separación permite que:

- Supabase maneje la autenticación (JWT, refresh tokens, expiración).
- La aplicación maneje los datos de dominio (matrícula, nombre, rol, equipo) sin depender del esquema de auth.

3.3 Flujo de autenticación

1. El usuario accede a `/login`.
2. Si ingresa credenciales, el formulario llama a `supabase.auth.signInWithPassword()`.
3. Si usa Google, se invoca `supabase.auth.signInWithOAuth()` con redirección a `/auth/callback`.
4. En `/auth/callback`, el Route Handler intercambia el código OAuth por una sesión Supabase y consulta `public.perfiles` para determinar el rol.
5. Si el usuario no tiene perfil, se cierra la sesión con `supabase.auth.signOut()` y se muestra un mensaje de error.
6. Si el perfil existe, se redirige al dashboard correspondiente: `/dashboard` para admin, `/equipo` para representante.

3.4 Roles del sistema

Cuadro 3.1: Roles de usuario

Rol	Usuario típico	Permisos
admin	Docente / Encargado del taller	Acceso total: CRUD de materiales, equipos y usuarios; KPIs; reportes; exportación CSV; creación y restablecimiento de contraseñas.
representante	Alumno designado como responsable de equipo	Solo su equipo: solicitar préstamos, devolver material, reportar consumos, ver historial.

Cada representante pertenece a un único equipo (`equipo_id` en `perfiles`) y es el único autorizado para realizar movimientos en nombre de ese equipo. Los administradores tienen `equipo_id = NULL`.

3.5 Protección de rutas por rol

El middleware de Next.js (`src/middleware.ts`) implementa la siguiente lógica:

1. **Sin sesión:** solo se permite acceso a `/login` y `/auth/*`. Cualquier otra ruta redirige a `/login`.
2. **Con sesión en `/login`:** redirige al dashboard según el rol.
3. **Admin en `/equipo/*`:** redirige a `/dashboard`.
4. **Representante en `/dashboard/*` o `/admin/*`:** redirige a `/equipo`.

Además del middleware, cada layout de dashboard verifica nuevamente el rol y redirige si no coincide. Esto constituye una defensa en profundidad.

3.6 Creación de usuarios

El flujo de creación desde la aplicación (`src/lib/actions/admin.ts`, `createUsuario`):

1. Verifica que el solicitante sea admin (`requireAdmin()`).
2. Llama a `supabase.auth.admin.createUser()` con email, contraseña y `email_confirm: true`.
3. Inserta el perfil en `public.perfiles` vinculado por `id`.
4. Si la inserción del perfil falla (por matrícula duplicada), elimina la cuenta auth creada para mantener consistencia.

3.7 Restablecimiento de contraseña

El admin puede restablecer la contraseña de cualquier usuario desde **Dashboard > Usuarios** mediante la Server Action `resetUsuarioPassword()`, que usa `supabase.auth.admin.updateUserById()` con la nueva contraseña (mínimo 6 caracteres).

3.8 Consideraciones de operación para Google OAuth

- Requiere registro de credenciales en Google Cloud Console y habilitación del proveedor en Supabase Auth.
- Las URLs de redirección autorizadas deben incluir tanto `http://localhost:3000/auth/callback` como la URL de producción.

- El login muestra un aviso de «¿Necesitas una cuenta?» para aclarar que Google no asigna permisos por sí solo: el perfil debe haber sido creado previamente por un administrador.

Flujos funcionales y reglas de negocio

4.1 Flujo de préstamo de material

1. **Seleccionar material:** el representante ve la lista de materiales disponibles con sus cantidades. La disponibilidad se calcula en tiempo real como `cantidad_total` menos préstamos activos pendientes.
2. **Indicar cantidad:** el representante elige la cantidad deseada mediante un campo numérico. La Server Action `pedirMaterial()` en `src/lib/actions/prestamos.ts` valida:
 - Que el usuario sea representante con equipo asignado (`requireRepresentante()`).
 - Que la cantidad sea un entero positivo (`getPositiveInt()`).
 - Que haya stock disponible suficiente (`validarCantidadDisponible()` de `src/lib/domain/inventory.ts`).
3. **Confirmar:** al enviar el formulario, la Server Action:
 1. Inserta un registro en `prestamos` con `estado = 'activo'`.
 2. Inserta un registro en `movimientos` con `tipo = 'salida_prestamo'`.
 3. Vincula el movimiento al préstamo mediante `movimiento_salida_id`.
 4. Ejecuta `revalidatePath` en `/equipo` y `/dashboard` para refrescar las vistas.

4.2 Flujo de devolución de material

1. **Ver préstamos activos:** el representante visualiza la lista de préstamos activos de su equipo. Cada préstamo muestra el material, la cantidad prestada y la cantidad pendiente por devolver.
2. **Seleccionar devolución:** el representante elige un préstamo y la cantidad a devolver. La Server Action `devolverMaterial()` valida:
 - Que el préstamo pertenezca al equipo del representante.
 - Que el préstamo esté en estado `'activo'`.
 - Que la cantidad a devolver no supere lo pendiente (`validarDevolucionPendiente()`).
3. **Confirmar:** al enviar:
 1. Se incrementa `cantidad_devuelta` en el préstamo.
 2. Si `cantidad_devuelta` alcanza o supera `cantidad_prestada`, el estado cambia a `'devuelto'`.
 3. Se inserta un registro en `movimientos` con `tipo = 'entrada_devolucion'`.
 4. Se ejecuta `revalidatePath`.

La devolución parcial está soportada: el representante puede devolver solo una parte del material prestado. El préstamo permanece activo hasta que la suma de devoluciones iguale o supere lo prestado.

4.3 Flujo de consumo de material

1. **Identificar material consumible:** el representante ve los materiales de tipo `consumible` disponibles para su equipo.
2. **Reportar cantidad consumida:** ingresa la cantidad. La Server Action `consumirMaterial()` valida:
 - Que el material sea de categoría `'consumible'` (`esConsumible()`).
 - Que haya stock disponible suficiente.

3. **Confirmar:** al enviar:

1. Se decrementa `cantidad_total` del material.
2. Se inserta un registro en `movimientos` con `tipo = 'consumo'`. No se crea registro en `prestamos`.
3. Se ejecuta `revalidatePath`.

Diferencia clave con el préstamo: el consumo descuenta directamente `cantidad_total` sin crear un préstamo, porque el material se gastó y no es retornable.

4.4 Flujo de entrada de stock (Admin)

1. El admin selecciona un material existente desde el catálogo.
2. Ingresa la cantidad a añadir. La Server Action `addMaterialStock()` valida que el material esté activo y la cantidad sea positiva.
3. Al confirmar:
 1. Se incrementa `cantidad_total` del material.
 2. Se inserta un registro en `movimientos` con `tipo = 'entrada_stock'`.
 3. Se ejecuta `revalidatePath`.

4.5 Creación y edición de materiales (Admin)

4.5.1 Creación

La Server Action `createMaterial()` en `src/lib/actions/admin.ts`:

1. Verifica que el solicitante sea admin.
2. Valida campos obligatorios (nombre, categoría, estado, cantidades no negativas).
3. Inserta el material en `public.materiales`.

4. Si se adjuntó una imagen, la sube a Supabase Storage (bucket `materiales`) y actualiza `imagen_url` e `imagen_path`.
5. Si la cantidad inicial es mayor a 0, registra un movimiento `entrada_stock`.

4.5.2 Edición

La Server Action `updateMaterial()` permite modificar `nombre`, `categoria`, `estado` y `stock_minimo`. `cantidad_total` solo se modifica mediante movimientos (`entrada_stock` o `consumo`), nunca por edición directa.

4.5.3 Imagen guía

Las Server Actions `updateMaterialImage()` y `deleteMaterialImage()` gestionan la imagen asociada al material:

- Las imágenes se almacenan en el bucket público `materiales` de Supabase Storage.
- Solo admins pueden subir, reemplazar o eliminar imágenes.
- El tamaño máximo es 5 MB. Formatos aceptados: JPEG, PNG, WebP, GIF.
- Al reemplazar o eliminar, se borra el archivo anterior del storage.

4.5.4 Eliminación lógica

La Server Action `deleteMaterial()` implementa eliminación lógica:

1. Verifica que el material esté activo y no tenga préstamos activos pendientes.
2. Si hay préstamos activos, rechaza la operación con un mensaje descriptivo.
3. Marca `activo = false` y establece `eliminado_at` con la fecha actual.
4. Inserta un movimiento `eliminacion_material` para trazabilidad.

4.6 Gestión de equipos (Admin)

- **Creación:** `createEquipo()` requiere solo el nombre.

- **Edición:** `updateEquipo()` modifica el nombre.
- **Eliminación:** `deleteEquipo()` verifica que el equipo no tenga representantes ni préstamos asociados antes de eliminar. Si tiene dependencias, rechaza la operación.

4.7 Gestión de usuarios (Admin)

- **Creación:** `createUsuario()` crea la cuenta en Supabase Auth e inserta el perfil. Si la inserción del perfil falla (matrícula duplicada), revierte la creación de la cuenta auth.
- **Edición:** `updatePerfil()` modifica nombre, matrícula, rol y equipo.
- **Restablecer contraseña:** `resetUsuarioPassword()` usa la API admin de Supabase Auth.
- **Eliminación:** `deletePerfil()` bloquea la auto-eliminación del admin activo y verifica que el usuario no tenga historial de préstamos o movimientos.

4.8 Reglas de negocio

- No se puede pedir más material del disponible.
- No se puede devolver más de lo pendiente.
- Solo materiales de tipo `consumible` pueden reportarse como consumo.
- Un equipo con historial o representantes no puede eliminarse.
- Un material con préstamos activos no puede eliminarse físicamente; se elimina de forma lógica.

5

Modelo de datos

El modelo de datos está compuesto por cinco tablas principales en el esquema `public` de PostgreSQL, más la tabla `auth.users` gestionada por Supabase Auth. El esquema se define en TypeScript mediante Drizzle ORM (`src/lib/db/schema.ts`) y se materializa mediante migraciones SQL en `supabase/migrations/`.

5.1 Diagrama Entidad-Relación

```
auth.users --1:1-- perfiles --N:1-- equipos
          |               |
          |               |
          v               v
    movimientos <----- prestamos
          |               |
          |               |
          v               v
    materiales <-----+
```

5.2 Tabla `perfiles`

Vinculada 1:1 con `auth.users`. Extiende los datos del usuario con información de dominio.

Cuadro 5.1: Columnas de `public.perfiles`

Columna	Tipo	Restricciones
<code>id</code>	UUID (PK)	FK → <code>auth.users.id</code>
<code>matricula</code>	TEXT	UNIQUE, NOT NULL
<code>nombre</code>	TEXT	NOT NULL
<code>rol</code>	TEXT	NOT NULL, CHECK IN ('admin', 'representante')
<code>equipo_id</code>	UUID	FK → <code>equipos.id</code> , NULLABLE
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>

Políticas RLS: los administradores ven todos los perfiles; los representantes solo ven el suyo.

5.3 Tabla equipos

Grupos de trabajo del taller. Cada equipo tiene un representante (alumno).

Cuadro 5.2: Columnas de `public.equipos`

Columna	Tipo	Restricciones
<code>id</code>	UUID (PK)	DEFAULT <code>gen_random_uuid()</code>
<code>nombre</code>	TEXT	NOT NULL
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>

Políticas RLS: admin ve y edita todos; representante solo ve su propio equipo.

5.4 Tabla materiales

Catálogo de materiales, herramientas, consumibles y equipos de protección.

Cuadro 5.3: Columnas de `public.materiales`

Columna	Tipo	Restricciones
<code>id</code>	UUID (PK)	DEFAULT <code>gen_random_uuid()</code>
<code>nombre</code>	TEXT	NOT NULL
<code>categoria</code>	TEXT	CHECK IN ('herramienta_manual', 'consumible', 'equipo_proteccion', 'equipo_medicion')
<code>cantidad_total</code>	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0
<code>stock_minimo</code>	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0
<code>estado</code>	TEXT	NOT NULL, DEFAULT 'bueno', CHECK IN ('bueno', 'desgastado', 'dañado')
<code>imagen_url</code>	TEXT	NULLABLE, URL pública en Supabase Storage
<code>imagen_path</code>	TEXT	NULLABLE, ruta en bucket para reemplazo o eliminación
<code>activo</code>	BOOLEAN	NOT NULL, DEFAULT <code>true</code> (eliminación lógica)
<code>eliminado_at</code>	TIMESTAMPTZ	NULLABLE, fecha de eliminación lógica
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>

Nota importante: la cantidad disponible **no se almacena** como columna. Se calcula en cada consulta como:

$$\text{cantidad_total} - \sum \text{préstamos activos.cantidad_prestada}$$

Esto evita inconsistencias entre el stock reportado y los préstamos reales. La lógica de cálculo reside en `src/lib/domain/inventory.ts` y en las consultas KPI de `src/lib/db/queries.ts`.

5.5 Tabla movimientos

Bitácora universal de trazabilidad. Es una tabla **inmutable**: solo se permiten inserciones (INSERT), nunca modificaciones (UPDATE) ni eliminaciones (DELETE).

Cuadro 5.4: Columnas de `public.movimientos`

Columna	Tipo	Restricciones
<code>id</code>	UUID (PK)	DEFAULT <code>gen_random_uuid()</code>
<code>tipo</code>	TEXT	CHECK IN ('entrada_stock', 'salida_prestamo', 'entrada_devolucion', 'consumo', 'eliminacion_material')
<code>material_id</code>	UUID	FK → <code>materiales.id</code> , ON DELETE RESTRICT
<code>cantidad</code>	INTEGER	CHECK: > 0 para todos los tipos excepto <code>eliminacion_material</code> (≥ 0)
<code>equipo_id</code>	UUID	FK → <code>equipos.id</code> , ON DELETE SET NULL
<code>usuario_id</code>	UUID	FK → <code>perfiles.id</code> , ON DELETE CASCADE
<code>prestamo_id</code>	UUID	FK → <code>prestamos.id</code> , ON DELETE SET NULL
<code>comentario</code>	TEXT	NULLABLE
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>

Políticas RLS: `admin` ve todos los movimientos; `representante` solo ve los de su equipo (`equipo_id`).

5.6 Tabla `prestamos`

Registro de préstamos activos e históricos, con soporte para devolución parcial.

Cuadro 5.5: Columnas de `public.prestamos`

Columna	Tipo	Restricciones
<code>id</code>	UUID (PK)	DEFAULT <code>gen_random_uuid()</code>
<code>equipo_id</code>	UUID	FK → <code>equipos.id</code> , ON DELETE CASCADE
<code>representante_id</code>	UUID	FK → <code>perfiles.id</code> , ON DELETE CASCADE
<code>material_id</code>	UUID	FK → <code>materiales.id</code> , ON DELETE RESTRICT
<code>cantidad_prestada</code>	INTEGER	NOT NULL, CHECK > 0
<code>cantidad_devuelta</code>	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0
<code>estado</code>	TEXT	NOT NULL, DEFAULT <code>'activo'</code> , CHECK IN (<code>'activo'</code> , <code>'devuelto'</code>)
<code>movimiento_salida_id</code>	UUID	FK → <code>movimientos.id</code> , ON DELETE SET NULL
<code>created_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT <code>now()</code>

Devolución parcial: cuando un representante devuelve menos de lo prestado, `cantidad_devuelta` se incrementa y el estado permanece `'activo'`. Al alcanzar o superar `cantidad_prestada`, el estado cambia a `'devuelto'`.

5.7 Relaciones entre tablas

Cuadro 5.6: Relaciones clave

Relación	Tipo	Explicación
<code>auth.users</code> → <code>perfiles</code>	1:1	Cada usuario <code>auth</code> tiene un perfil de dominio.
<code>perfiles</code> → <code>equipos</code>	N:1	Varios representantes pertenecen a un equipo. Admin tiene <code>NULL</code> .
<code>equipos</code> → <code>prestamos</code>	1:N	Un equipo tiene muchos préstamos.
<code>materiales</code> → <code>prestamos</code>	1:N	Un material puede estar en muchos préstamos.
<code>materiales</code> → <code>movimientos</code>	1:N	Trazabilidad completa de cada material.
<code>prestamos</code> → <code>movimientos</code>	1:N	Un préstamo tiene movimientos asociados (salida, devoluciones).

5.8 Categorías de materiales

Cuadro 5.7: Categorías disponibles

Categoría	Clave	Ejemplos
Herramienta manual	<code>herramienta_manual</code>	Pinza de corte, martillo, perica, cortatubos
Consumible	<code>consumible</code>	Cilindro de propano, gas MAP, boquillas
Equipo de protección	<code>equipo_proteccion</code>	Guantes de seguridad, lentes, careta
Equipo de medición	<code>equipo_medicion</code>	Manómetros, termómetros, multímetros

5.9 Estados de material

- **bueno:** material en condiciones óptimas de uso.
- **desgastado:** material funcional pero con signos de uso.
- **dañado:** material que requiere reparación o reemplazo.

5.10 Tipos de movimiento

- `entrada_stock`: incremento de `cantidad_total` por el admin.
- `salida_prestamo`: préstamo solicitado por un representante.
- `entrada_devolucion`: devolución (total o parcial) de un préstamo.
- `consumo`: material consumible agotado, reportado por representante. No genera préstamo.
- `eliminacion_material`: marcador de trazabilidad cuando un material es eliminado lógicamente.

5.11 Stock semilla

El archivo `src/lib/db/seed.ts` contiene los datos iniciales para desarrollo local. Credenciales de prueba:

Cuadro 5.8: Credenciales seed para desarrollo local

Rol	Correo	Contraseña
Admin	<code>admin@cecyte.edu.mx</code>	<code>Admin123!</code>
Representante 1	<code>rep1@cecyte.edu.mx</code>	<code>Rep12345!</code>
Representante 2	<code>rep2@cecyte.edu.mx</code>	<code>Rep12345!</code>

6

Supabase, migraciones, operación y pruebas

6.1 Supabase local

La configuración local vive en `supabase/config.toml`. Ahí se definen puertos, Auth, Realtime, Storage y el sitio base para callbacks. Las migraciones SQL están en `supabase/migrations`.

6.2 Migraciones

- `0001_schema.sql`: esquema base, índices y triggers `updated_at`.
- `0002_rls.sql`: políticas RLS para admin y representante.
- `0003_materiales_soft_delete.sql`: soft delete y nuevo tipo de movimiento.
- `0004_material_images.sql`: imágenes guía y bucket de Storage.

6.3 Supabase Storage

Los materiales pueden tener imagen guía opcional. El bucket `materiales` es público y acepta JPEG, PNG, WEBP y GIF, con límite de 5 MB.

6.4 Variables de entorno

Cuadro 6.1: Variables de entorno requeridas

Variable	Ámbito	Descripción
NEXT_PUBLIC_SUPABASE_URL	público	URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_PUBLIC_KEY	público	Clave anónima/publicable
SUPABASE_SECRET_KEY	servidor	Clave <i>service_role</i> (nunca al cliente)
DATABASE_URL	servidor	Conexión directa para migraciones Drizzle
GOOGLE_CLIENT_ID	servidor	Client ID de Google Cloud OAuth
GOOGLE_CLIENT_SECRET	servidor	Client Secret de Google Cloud OAuth

El código mantiene compatibilidad con las variables legacy locales `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY` como fallback.

6.5 Scripts operativos

- `corepack pnpm dev`: arranque local.
- `corepack pnpm build`: compilación de producción.
- `corepack pnpm verify`: lint + typecheck + tests + build.
- `corepack pnpm test`: pruebas unitarias.
- `corepack pnpm test:e2e`: pruebas end-to-end.

6.6 Seed y carga inicial

El seed crea equipos, usuarios y materiales base del taller. También existe una carga masiva de inventario en `scripts/` con versión SQL y versión CJS.

6.7 Pruebas

- Vitest valida la lógica pura de inventario.

- Playwright cubre login, redirecciones por rol y flujos de acciones.
- El pipeline de verificación intenta detectar errores antes del despliegue.

6.8 Reportes (Admin)

La vista `/dashboard/reportes` permite:

- Filtrar movimientos por período (hora, día, semana, mes), tipo de movimiento, equipo y material.
- Visualizar resultados paginados con columnas: fecha, tipo, material, cantidad, equipo, usuario, comentario.
- Exportar a CSV mediante una Server Action que genera el archivo en el servidor.

UI/UX y diseño

7.1 Identidad visual institucional

El diseño se basa en los colores institucionales del CECYTE, definidos en el tema extendido de Tailwind (`tailwind.config.ts`):

Cuadro 7.1: Paleta de colores CECYTE

Color	Hex	Uso
Verde CECYTE primario	#006341	Sidebar, header, botones principales
Verde CECYTE oscuro	#004d35	Hover states
Verde CECYTE claro	#e6f0ec	Fondos suaves, badges
Naranja acento	#F97316	Alertas de stock bajo
Rojo	#EF4444	Errores, materiales agotados, acciones destructivas
Verde claro	#22C55E	Confirmaciones, material disponible

7.2 Stack de diseño

Cuadro 7.2: Herramientas de diseño

Elemento	Herramienta
Framework CSS	Tailwind CSS 3.4
Componentes UI	shadcn/ui (base-nova, Radix)
Iconos	Lucide React (1.14.0)
Gráficos	Recharts (3.8.1)
Tipografía	Inter (variable font)
CSS Variables	shadcn/ui theming system

7.3 Directrices de interfaz

7.3.1 Botones y acciones

- Altura mínima de 48px para accesibilidad táctil.
- Color por función semántica:
 - Verde institucional: acciones principales (crear, confirmar, pedir).
 - Gris: acciones secundarias (cancelar, volver).
 - Rojo: acciones destructivas (eliminar).
- Iconos siempre acompañados de texto hasta que el usuario se familiarice con la interfaz.
- Confirmación visible tras cada acción exitosa.

7.3.2 Formularios

- Etiquetas explícitas en todos los campos (no depender de placeholders).
- Campos numéricos con etiquetas como «*Cantidad inicial*» y «*Stock mínimo*», no solo valores por defecto.
- Campo de contraseña sin placeholder engañoso de puntos cuando está vacío.

- Botón para mostrar/ocultar contraseña en el login.
- Validación tanto client-side como server-side con mensajes de error descriptivos.

7.3.3 Regla de 3 clics

Toda operación del representante debe completarse en 3 acciones o menos:

- **Pedir:** ver materiales → elegir cantidad → confirmar.
- **Devolver:** ver préstamos activos → seleccionar → confirmar.
- **Consumir:** ver consumibles → reportar cantidad → confirmar.

7.3.4 Diseño responsive

- **Mobile-first:** diseñado para pantallas desde 375px (teléfonos de los alumnos).
- Layout single column en móvil, 2 columnas en tablet, 3+ columnas en desktop.
- **Admin:** sidebar fija en desktop (`md:w-64`), bottom tab bar en móvil (5 pestañas: Dashboard, Materiales, Equipos, Usuarios, Reportes).
- **Representante:** header sticky con nombre del equipo, bottom tab bar en móvil (Pedir, Devolver).
- Tablas de datos que se convierten en cards apiladas en pantallas pequeñas.

7.4 Distribución del Dashboard (Admin)

El panel principal del administrador (`/dashboard`) organiza 9 KPIs en cards y gráficos:

Dashboard					[Período v]
KPI 1	KPI 3	KPI 7			
Material	Stock	Tasa	KPI 2		
total	Bajo	devolución	Prestado vs		
			Disponible		

KPI 6 - Movimientos por período	
(Gráfico de líneas)	
+-----+-----+	
KPI 4	KPI 5
Top 5 materiales	Equipos más activos
(Barras horizontal)	(Barras vertical)
+-----+-----+	
KPI 9 - Consumibles agotados (Tabla)	
+-----+-----+	
KPI 8 - Estado de materiales	
(Gráfico de anillo)	
+-----+-----+	

7.5 Panel del Representante

El panel del representante (`/equipo`) muestra:

- Tarjeta superior con nombre del representante y equipo.
- Catálogo de materiales disponibles con cantidades en tiempo real.
- Sección de préstamos activos de su equipo con acciones de devolución.
- Botones para solicitar préstamo y reportar consumo.
- Indicador «*Inventario en vivo*» que muestra el estado de la conexión Realtime.

7.6 Actualización en tiempo real

El componente `RealtimeRefresh` (`src/components/realtime-refresh.tsx`):

- Es un Client Component que se suscribe a cambios en las tablas `materiales` y `prestamos` mediante Supabase Realtime.
- Cuando detecta un cambio (INSERT, UPDATE, DELETE), llama a `router.refresh()` para que Next.js revalide los Server Components.
- Muestra un indicador visual de estado de la conexión y un botón de refresco manual.

- Aparece tanto en el layout del admin como en el del representante.

7.7 Formulario de login

La página `/login` incluye:

- Campos de correo y contraseña con validación client-side.
- Botón para mostrar/ocultar contraseña (icono de ojo).
- Botón «*Continuar con Google*» para OAuth institucional.
- Aviso «*¿Necesitas una cuenta?*» que indica que Google no asigna permisos automáticamente.
- Mensajes de error descriptivos para credenciales inválidas y cuentas sin perfil.

7.8 Decisiones UX relevantes

- Retroalimentación visual inmediata en formularios.
- Navegación móvil específica por rol.
- Búsquedas tolerantes a acentos y mayúsculas.
- Tablas y tarjetas simples para lectura rápida.
- Actualización en vivo para evitar estados obsoletos.

7.9 Componentes reutilizables

Cuadro 7.3: Componentes reutilizables principales

Componente	Descripción
ManagedForm	Envoltorio para Server Actions con feedback visual.
SearchBox	Búsqueda unificada con conteo de resultados y debounce de 300ms.
RealtimeRefresh	Refresco automático por cambios en tiempo real vía Supabase Realtime.
KpiCard	Tarjeta KPI de alto contraste.
PeriodSelector	Selector de período para el dashboard.
Gráficos	Barras, líneas y donuts con Recharts.

Componentes

El proyecto sigue un patrón de componentes organizados en tres niveles:

1. **Componentes UI base** (`src/components/ui/`): primitivas de shaden/ui generadas mediante el CLI.
2. **Componentes de dominio** (`src/components/dashboard/`, `src/components/forms/`): componentes específicos del negocio.
3. **Componentes de infraestructura**: utilidades transversales como refresco en tiempo real y búsqueda.

8.1 Componente RealtimeRefresh

- **Ubicación:** `src/components/realtime-refresh.tsx`.
- **Tipo:** Client Component (`'use client'`).
- **Props:**
 - `showStatus?: boolean` — muestra indicador de conexión activa.
 - `label?: string` — texto opcional del botón.
- **Funcionamiento:**
 1. Crea un canal de Supabase Realtime suscrito a cambios en las tablas

`materiales` y `prestamos`.

2. Al recibir un evento de cambio, invoca `router.refresh()` del App Router.
 3. Esto fuerza que los Server Components se re-ejecuten y obtengan datos frescos.
- Se integra en ambos layouts: `(dashboard)/layout.tsx` y `(equipo)/layout.tsx`.

8.2 Componente SearchBox

- **Ubicación:** `src/components/search-box.tsx`.
- **Tipo:** Client Component.
- **Props:**
 - `placeholder?: string` — placeholder del campo de búsqueda.
- **Funcionamiento:**
 1. Lee el valor inicial de `URLSearchParams`.
 2. Implementa debounce de 300ms para evitar peticiones excesivas.
 3. Al cambiar el valor, actualiza `searchParams` en la URL, lo que dispara un re-render del Server Component padre.
- Se usa en catálogo de materiales, lista de equipos y lista de usuarios.

8.3 Componentes de formularios

Ubicados en `src/components/forms/`, incluyen formularios reutilizables para:

- Creación y edición de materiales.
- Creación y edición de equipos.
- Creación y edición de usuarios.
- Restablecimiento de contraseña.

- Entrada de stock.
- Solicitud de préstamo.
- Devolución de material.
- Reporte de consumo.

Cada formulario sigue el patrón de Server Actions: el `action` del formulario apunta directamente a la función exportada con `'use server'`.

8.4 Componentes del dashboard

Ubicados en `src/components/dashboard/`, incluyen:

- Gráficos Recharts (barras, líneas, anillos) para cada KPI.
- Cards de KPIs con número grande, etiqueta e icono.
- Tablas de datos con paginación y ordenamiento.
- Indicadores de stock bajo con color condicional.

8.5 Componentes UI (shadcn/ui)

Generados mediante `npx shadcn@latest add` y ubicados en `src/components/ui/`. Incluyen:

- `button.tsx` — variantes: default, destructive, outline, secondary, ghost, link.
- `card.tsx` — Card, CardHeader, CardTitle, CardDescription, CardContent, CardFooter.
- `input.tsx` — campo de texto estilizado.
- `label.tsx` — etiqueta accesible.
- `dialog.tsx` — modal accesible basado en Radix.
- `select.tsx` — selector desplegable.
- `table.tsx` — tabla con header, body, row, cell.

- `badge.tsx` — etiqueta de estado con variantes de color.
- `dropdown-menu.tsx` — menú contextual.
- `skeleton.tsx` — placeholder de carga.
- `toast.tsx` / `toaster.tsx` — notificaciones toast.

Todos heredan el tema de shadcn/ui configurado en `components.json` (estilo base-nova, colores neutral, CSS variables).

8.6 Clientes Supabase

Aunque no son componentes React, los clientes Supabase son utilidades reutilizables clave:

- `src/lib/supabase/server.ts` — exporta dos funciones:
 - `createClient()`: cliente con cookies de sesión para Server Components, Server Actions y Route Handlers. Lee y escribe cookies mediante `next/headers`.
 - `createAdminClient()`: cliente con `service_role` para operaciones privilegiadas. No usa cookies de usuario para que la sesión autenticada no reemplace el header de autorización y bloquee RLS.
- `src/lib/supabase/client.ts` — cliente para el navegador usando `createBrowserClient` de `@supabase/ssr`.

8.7 Consideraciones de diseño

- **Server Components por defecto:** la mayoría de las páginas son Server Components que consultan Supabase directamente. Solo los componentes que necesitan interactividad (Realtime, búsqueda) son Client Components.
- **Separación de lecturas y escrituras:** las lecturas usan `createClient()` (respetando RLS); las escrituras usan `createAdminClient()` (`service_role`, evita bloqueos de RLS). Esto permite que las políticas RLS protejan las lecturas mientras las mutaciones complejas se manejan con lógica de validación en las Server Actions.
- **Revalidación explícita:** cada Server Action llama a `revalidatePath()` para las rutas afectadas, asegurando que los Server Components se re-ejecuten tras

cada mutación.

9.1 Observaciones importantes

- El proyecto depende de Supabase Realtime para refrescar datos sin polling manual.
- Las mutaciones administrativas usan service role y deben mantenerse aisladas del cliente.
- La documentación Markdown existente en [docs/](#) sigue siendo útil como respaldo temático.

9.2 Decisiones de diseño del documento

- Documento unificado a partir de las secciones previamente modulares. La fusión preserva el contenido completo de las versiones detalladas (`02_stack_arquitectura.tex`, `03_estructura_proyecto.tex`, `04_autenticacion_roles.tex`, `05_modelo_datos.tex`, `06_flujos_funcionales.tex`, `07_ui_ux.tex`, `08_componentes.tex`) junto con los combinados originales.
- Compilación con pdfLaTeX (requiere los paquetes estándar de TeX Live: `lmodern`, `microtype`, `titlesec`, `listings`, `tcolorbox`, `fancyhdr`).
- Los nombres de archivo y código se renderizan con `listings` en modo inline; las rutas largas dentro de tablas pueden requerir ajuste manual si el ancho de columna resulta insuficiente.